

0.3.2. Écrire au même endroit écrase — doublé, deux positions

L'écrasement à deux places : en haut à gauche ET au milieu, le B écrase le A.

1. Ce qu'il faut comprendre

C'est le 0.3 et le 0.3.1 réunis : les deux écrasements, chacun à sa place.

Chaque case est indépendante : ce qui s'écrit en (0, 0) ne touche pas (10, 8). Aux deux endroits, le B écrase le A.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **255 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         51 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         32 o ← CopierTuiles - 2 lettres sur 43
"A"            1 o ← texte() - un texte gravé
"B"            1 o ← texte() - un texte gravé
"A"            1 o ← texte() - un texte gravé
"B"            1 o ← texte() - un texte gravé
```

Lettres gravées : AB

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 4    44 o sur place, dans main() + EcrireTexte (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte> // écrit un texte à l'écran
```

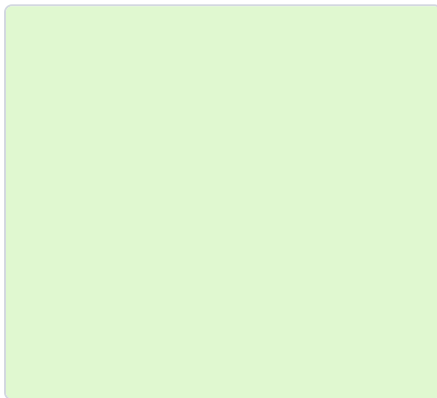
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(0, 0, "A");    // en haut à gauche (le 0.3)...
    texte(0, 0, "B");    // ...écrasé par le B
    texte(10, 8, "A");   // au milieu (le 0.3.1)...
    texte(10, 8, "B");   // ...écrasé par le B

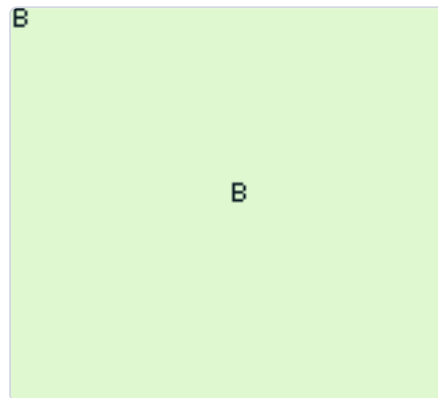
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Deux B : en haut à gauche et au milieu. Aucun A.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ un B en haut à gauche

✓ un B au milieu

Ensuite : **0.4. L'alphabet à la main, jusqu'au bout de la ligne** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++